DSphinx: A Decentralized Path Selection for Decentralized Networks

Aurélien Chassagne[1 [0009−0008−7207−868X]], Manuel Mota Leal Dias[1], Iness Ben Guirat[1 [0000−0002−8766−594X]], Jan Tobias Mühlberg[1 [0000−0001−5035−0576]], and Jean-Michel Dricot[1 [0000−0002−8539−9940]]

Université Libre de Bruxelles (ULB), Brussels, Belgium

Abstract Sphinx packet is the foundation of many privacy-preserving decentralized systems, including mixnets and the Lightning Network, providing strong guarantees of unlinkability, confidentiality, and integrity. However, because clients independently select routing paths, malicious clients can deviate from the prescribed routing algorithm to launch attacks such as route biasing, jamming, and targeted deanonymization.
We present *DSphinx*, a decentralized path-selection protocol that distributes route sampling across multiple semi-trusted third parties. Using threshold secret sharing and elliptic-curve cryptography, DSphinx prevents clients from arbitrarily selecting routes while preserving the compact packet structure and security guarantees of Sphinx. We provide a formal security analysis showing that DSphinx preserves unlinkability, integrity protection, replay resistance, and collusion resistance under standard cryptographic assumptions. A prototype implementation demonstrates that the additional computational overhead remains modest, introducing less than 50 ms of client-side latency. Our approach reduces client control over route selection while maintaining the efficiency and privacy guarantees of Sphinx.

Keywords: Sphinx · Decentralized networks · Mixnet · Lightning.

1 Introduction

Privacy at the network layer, such as in mixnet-based systems [6–9,14,18,19,21] for communication and in the Lightning Network [22] for financial transactions, is achieved through layered encrypted packets that traverse multiple nodes, with each node decrypting one layer and forwarding the packet to the next hop. To prevent malicious routing nodes and network adversaries from de-anonymizing traffic at the network layer, each intermediary node learns only its immediately preceding and succeeding nodes, not the full path nor its position within it. Additionally, packets are padded to a fixed size and processed such that their format reveals no linkable information across hops, preventing traffic correlation based on size or format. At the core of these guarantees is Sphinx [10], a compact fixed-size packet format that provides cryptographically strong unlinkability and per-hop integrity protection [10]. The Sphinx packet format, introduced by

Danezis and Goldberg [10] in 2009, has become a foundational component of modern anonymous communication systems such as the Lightning Network [22], Nym [11], and Katzenpost [15].

However, despite its strengths, the Sphinx packet format is constructed entirely by clients before entering the network, meaning that clients retain full control over route selection and can easily deviate from the prescribed routing algorithm, making the system vulnerable to attacks by malicious clients. For example, in mixnets, a client may bias route selection by violating the routing algorithm, thereby reducing the number of messages mixed at a specific node in order to target an honest client or degrade the system's overall anonymity [3].

In the Lightning Network, a malicious client can launch a jamming attack by sending payments through honest nodes and deliberately never resolving them, locking their channels and freezing liquidity. Additionally, by repeatedly routing through specific nodes and observing response times, the attacker can infer their current payment load and activity patterns [16, 23].

In this work, we introduce DSphinx, a *decentralized Sphinx path-selection scheme* in which multiple independent semi-trusted third parties (STTPs) collaboratively derive the packet's route from a client-provided nonce.

Decentralizing route selection while preserving Sphinx's guarantees raises several challenges. First, the protocol must prevent individual STTPs, or coalitions of corrupted STTPs, from learning routing information or recovering cryptographic secrets. Second, routing nodes should learn no more than in the original Sphinx design: each node learns only its own shared secret and the identity of the previous and next hop. Third, unlinkability must be preserved against a global passive adversary capable of observing all network traffic. Finally, decentralization must not introduce weaknesses that enable adversaries to bias sampling or correlate packets across hops.

DSphinx addresses these challenges by combining threshold secret sharing with elliptic-curve cryptography to decentralize route sampling while preserving the compact packet format and per-hop processing model of Sphinx. Route selection is deterministically derived from a client-provided nonce and collaboratively computed by STTPs, preventing clients from arbitrarily selecting routes. To the best of our knowledge, DSphinx is the first work to address malicious clients in privacy-preserving networks by proposing a decentralized route selection. By reducing client control over path selection while preserving Sphinx's efficiency and security properties, DSphinx represents a practical step toward more robust anonymous communication systems.

In summary, our main contributions are as follows:

- We design *DSphinx*, the first *decentralized Sphinx* path-selection scheme that prevents malicious clients from deviating from the routing protocol.
- We provide a detailed security analysis of *DSphinx*, considering passive and active network adversaries, corrupted routing nodes, and colluding STTPs.
- We release a benchmark[1] for measuring latency and computational overhead, implemented in both Rust and Python.

[1] `https://github.com/AurelienCha/Decentralized-Sphinx.git`

2 Background and Related Work

2.1 Decentralized Networks

Since Chaum's seminal work on untraceable email in 1981 [6], there has been a great amount of research related to mixnet design [2, 6–9, 14, 18, 19, 21]. A mixnet is a network of nodes that delay and shuffle packets as they traverse a sequence of hops before reaching their final destination, preventing adversaries from correlating input and output traffic.

Mixnets have been known to resist powerful adversaries typically called global passive adversaries whose goal is to deanonymize clients and trace message through traffic analysis techniques. Mixnets achieve this by (i) mixing messages inside the mixnodes, thereby hiding the connection between incoming and outgoing packets and (ii) the use of a cryptographic packet format that prevents adversaries from tracing messages based on packet size or format. Messages are first divided into fixed-size packets to prevent size-based correlation attacks. Each packet is then encrypted in multiple layers, together with the necessary routing information. This guarantees that every mixnode learns only its immediate predecessor and successor along the path.

The Lightning Network [22] is a payment-channel that allow two users to send off-chain transactions without continuously interacting with the Bitcoin blockchain. In this design, even users who do not have a direct channel may still transact by routing payments through intermediate peers. When a node wants to pay another node without having a direct channel, the transaction originator first computes the shortest path to the recipient and then encrypts the routing information multiple times, where each cryptographic layer corresponds to a hop along the path. Although the Lightning Network was initially introduced to address Bitcoin's scalability, it also addresses the privacy issues introduced by peer-to-peer networks [24]. In particular adversaries should not be able to identify the sender or receiver of a transaction. Additionally an intermediate node should not know whether their predecessor is the transaction's originator or whether their successor is the transaction's recipient.

These privacy guarantees, both in mixnets and in the Lightning Network, are achieved using the Sphinx packet format [10].

2.2 Sphinx Packet Format

The *Sphinx* packet format provides three essential security properties: per-hop confidentiality, per-hop integrity protection, and strong unlinkability.

First, Sphinx ensures *per-hop confidentiality* by encrypting routing information in layers, similar to classical onion routing. Each mixnode learns only its immediate predecessor and successor, but gains no knowledge about the overall path, the sender, or the final destination. Second, Sphinx provides *per-hop integrity protection* through a cryptographic tag embedded in each layer of the header. These tags prevent adversaries from tampering with routing information. Any modification results in an integrity-tag verification failure at the routing

node, causing the packet to be discarded. This ensures that adversaries cannot tamper with routes or perform active attacks such as header tagging in order to target specific messages or clients. Third, Sphinx offers *strong unlinkability* by ensuring that packets entering and leaving a mixnode are computationally indistinguishable. This is achieved by using fixed-size headers and payloads and by re-randomizing cryptographic elements at every hop. Each hop transforms the packet into a fresh, uniformly distributed header, preventing an adversary from linking an incoming packet to its outgoing counterpart. These guarantees have made Sphinx the de facto standard for privacy-preserving routing in modern decentralized systems such as the Lightning Network [22] and mixnet-based architectures like Nym [11] and Katzenpost [15].

However, in all these systems, the Sphinx packet format must be fully constructed by the client, which introduces vulnerabilities when malicious clients deviate from the prescribed routing algorithm. Routing algorithms in decentralized networks are chosen carefully balancing latency with strong privacy guarantees. Preventing malicious clients from deviating from the routing algorithm, without relying on a central authority and while preserving unlinkability between packets and clients has been a challenging open problem. A malicious client can intentionally deviate from the routing protocol to launch attacks, either against the functioning of the system, against specific clients, or against the overall privacy provided by the system. For example, in the Lightning Network, a malicious client can route payments through long paths specifically to launch jamming attacks, freeze funds, or infer activity patterns of nodes through response times [16, 23]. In Nym, nodes should be chosen uniformly across layers to maximize anonymity [3]; therefore, an attacker can bias route selection either to reduce the overall anonymity of the system or to target specific clients and messages.

3 Threat Model and Security Properties

In this section, we present our threat model and define the privacy and the desired security properties of our system. As illustrated in Fig. 1, the system involves three types of entities: clients, a set of semi-trusted third parties (STTPs), and network nodes. Clients delegate route sampling to the STTPs, which jointly derive the routing path and secret shares associated with the selected nodes. The client subsequently reconstructs the route and the corresponding per-hop shared secrets from a threshold number of STTP responses. Clients are assumed to be malicious, meaning that they may deviate from the protocol to bias path selection or perform manipulation-based routing attacks. STTPs and network nodes are assumed to be *honest-but-curious*: they correctly execute the protocol but are not trusted for privacy guarantees. Security is ensured as long as fewer than a threshold d of STTPs collude.

3.1 Threat Model

We consider malicious client, corrupted network nodes and STTPs, as well as global passive and active network adversaries. We assume the hardness of the Elliptic Curve Discrete Logarithm Problem (ECDLP) in the underlying group. Hash functions are modeled as random oracles, and HMAC is assumed to be existentially unforgeable under chosen-message attacks. We further assume that network nodes anonymously submit their secret shares to the STTPs during the setup phase, and less than a threshold number d of STTPs may be corrupted or colluding.

Malicious Client. A malicious client may attempt to bias or manipulate decentralized route sampling to favor specific nodes or paths.

Corrupted Network Node. A corrupted node may reveal its entire internal state, including private keys and shared secrets. Compromising a node must not compromise the privacy or integrity of the remaining path, and unlinkability is preserved as long as at least one node on the path is honest.

Corrupted STTP. Corrupted STTPs may attempt to infer additional information from their computation. We allow collusion among STTPs, provided that fewer than d STTPs are corrupted. Under this assumption, colluding STTPs cannot reconstruct complete routes or derive shared secrets.

Passive Network Adversary. A passive adversary can observe traffic at any point in the network but cannot modify, inject, or drop packets. Its goal is to break unlinkability by correlating incoming and outgoing packets.

Active Network Adversary. An active adversary can arbitrarily modify, drop, replay, or inject packets. Its objective is to forge valid integrity tags to tamper with headers while remaining undetected.

3.2 Security and Privacy Objectives

Our DSphinx construction aims to preserve the security guarantees of Sphinx [10], including unlinkability and integrity protection, while extending them to a decentralized route sampling setting.

Unlinkability. We consider *node-level unlinkability*, requiring that an adversary cannot link an incoming packet to its corresponding outgoing packet at any node. This is achieved if all per-hop header components are computationally unlinkable without knowledge of the node's secret key.

Integrity-Forgery Resistance. As in Sphinx, DSphinx employs per-hop integrity tags to ensure that any modification of packet headers can be detected by honest nodes. An active adversary must not be able to alter a packet in transit while producing a valid integrity tag.

Collusion Resistance. We require that collusions between clients, STTPs, and network nodes reveal no additional routing or anonymity information beyond their individual views. In particular, as long as fewer than d STTPs are corrupted, adversaries cannot reconstruct complete routing information or compromise the security of honest participants.

4 Protocol Description

The proposed decentralized protocol distributes path sampling across a threshold number of semi-trusted third parties (STTPs), in order to mitigate client-side route biasing attacks. Fig. 1 illustrates the protocol, which is divided into three phases: a one-time setup phase (Step 0), a decentralized path sampling phase (Steps 1-3), and a header construction phase (Step 4).

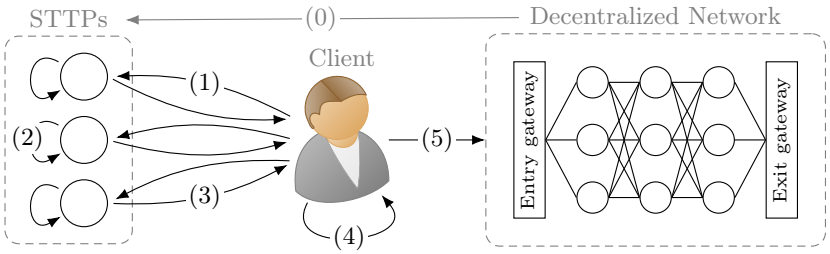


Figure 1: Overview of decentralized path selection. (0) Setup, (1) client nonce, (2) path sampling, (3) aggregation, (4) header construction, (5) packet forwarding.

DSphinx header. The current protocol is based on an adaptation of the Sphinx packet format that we called DSphinx. The key distinction is that each header element is independently encoded as an elliptic curve (EC) point. Consequently, the DSphinx header is structured as a sequence of fixed-size chunks, as illustrated in Fig. 2, where each chunk corresponds to a single header element represented by an EC point. Each network node along the packet route contributes for two chunks: one encoding its address and another encoding a per-hop integrity tag. Therefore, a route of length m is represented by $n = 2m + 1$ chunks, accounting for two chunks per network node and one additional chunk corresponding to the final destination. Some routing information, such as IP addresses, must be reversibly encoded as elliptic curve points. To this end, DSphinx uses the Elligator mapping [5], which produces points computationally indistinguishable from randomly sampled group elements.

Design rationale. DSphinx decentralizes route sampling across multiple STTPs to prevent both clients and STTPs from manipulating path selection. Clients contribute only a nonce w, which is deterministically mapped to the curve point α using a hash-to-curve function. The routing path is then deterministically derived from the tuple $(\alpha, \text{timestamp}, i)$, where the timestamp ensures different sampled paths over time, and the hop index i ensures independent node selection across hops. Since hash-to-curve is non-invertible, influencing the sampled route reduces to repeated trial over the nonce space. Likewise, all honest STTPs execute the same deterministic procedure and therefore produce identical routing decisions from the same nonce. Any deviation results in inconsistent shares that cannot be reconstructed by the client, which queries at least d STTPs and

reconstructs the route from any consistent subset of d responses. Furthermore, STTPs observe only node pseudonyms and therefore cannot associate sampled routes with physical nodes without additional collusion.

4.1 Setup

Each network node i samples a random identifier r_i and constructs two random polynomials f_i and g_i of degree $d-1$, such that: $f_i(0) = k_i$ and $g_i(0) = N_i$, where k_i denotes the node's secret key, N_i its encoded address in $\mathbb{G}$, and d the reconstruction threshold.

Each STTP is identified by a value t, computed as the hash of its address. For each STTP t, network node i evaluates $f_i(t)$ and $g_i(t)$ and securely transmits the corresponding shares together with r_i to the STTP t as in (1). We assume these shares are transmitted anonymously, preventing STTPs from linking them to specific nodes. Each STTP stores the received shares indexed by r_i, which later serves to pseudorandomly select network nodes during route sampling.

$$(r_i, \quad k_{it} = f_i(t), \quad N_{it} = g_i(t)) \tag{1}$$

Adding a new node only requires executing the setup procedure for that node. Adding a new STTP requires each node to compute and distribute an additional polynomial evaluation. However, changing the threshold d requires reinitializing the entire setup.

4.2 Path Sampling

To sample a routing path, the client sends a nonce w to at least d STTPs. Each STTP maps this nonce to the curve point α using a hash-to-curve function.

For each hop i, the STTP computes $r'_i = \text{hash}(\alpha, \text{timestamp}, i)$, and selects the network node share (r_i, k_{it}, N_{it}) whose identifier r_i is closest to r'_i. The STTP then computes the partial Diffie-Hellman shared secret $S_{it} = k_{it}\alpha$, and returns its identifier together with the selected shares and partial shared secrets to the client:

$$\left(t,\ \{N_{it}\}_{i=1}^{m},\ \{S_{it}\}_{i=1}^{m}\right)$$

Note that the path sampling scheme proposed in this work is designed for uniform random selection of routing nodes. While the Lightning Network implements variants of shortest-path algorithms such as LND [20], eclair [1] and c-lightning [13], prior research has shown that random route selection offers stronger privacy guarantees [17, 24, 25]. Similarly, in mixnet-based systems, the routing strategy that maximizes privacy is uniform random selection of mixes across layers [3, 4]. Hence, this work focuses on random path sampling, leaving alternative routing algorithms for future work.

4.3 Aggregation

Upon receiving responses from at least d STTPs, the client reconstructs node addresses and shared secrets via Lagrange interpolation as in (2), where $\lambda(t)$ denotes the Lagrange coefficient for STTP identifier t. Since all STTPs process the same nonce w, the reconstructed values correspond to a consistent path.

$$N_i = \sum_t \lambda(t) N_{it}, \qquad S_i = \sum_t \lambda(t) S_{it}, \tag{2}$$

To prevent linkability, the client samples a random scalar r to rerandomize the shared secrets and updates the cryptographic element α. Those shared secrets are derived iteratively in cascade, where each hop incorporates previous secrets:

$$s_i = \mathrm{HashToScalar}\left(rS_i \prod_{j=1}^{i-1} s_j \right).$$

4.4 Header Construction

Given the reconstructed route $\{N_i\}_{i=1}^m$ and per-hop shared secrets $\{s_i\}_{i=1}^m$, the client constructs the DSphinx header for destination Δ (encoded in $\mathbb{G}$). The header construction proceeds in two phases; a preprocessing phase (Alg. 1, Fig. 2a) that generates filler chunks, followed by a layer-by-layer encryption phase (Alg. 2, Fig. 2b) applied in reverse order, from the last hop to the first. To ensure unlinkability, encryption is performed using a set of independently derived generators $\{G_j\}_{j=1}^n$, one per chunk in the header, such that no scalar relationship between any pair of generators is known.

Preprocessing Phase. The preprocessing phase prepares a vector of filler chunks ϕ according to (3). These filler chunks ensure that after each encryption layer the last two chunks evaluate to the identity element and can thus be safely truncated, ensuring a constant header size and reversibility when processed by the network nodes. The detailed procedure is specified in Alg. 1, which takes as input the set of shared secrets $\{s_i\}$ and the set of chunk generators $\{G_j\}$, and outputs $n-1$ filler chunks, where $n = 2m+1$ is the header size for a path of length m. An example for $m = 3$ is illustrated in Fig. 2a.

$$\phi_j = -\sum_{i=1}^{m} s_i \, G_{\,n+j-2i}, \qquad 1 \le j \le n-1. \tag{3}$$

Layered Encryption Phase. The header is initialized by concatenating the destination chunk Δ with the filler chunks ϕ, producing a n-chunks header. Encryption is then applied iteratively in reverse path order, from $i = m$ down to $i = 1$, as illustrated in Fig. 2b and detailed in Alg. 2.

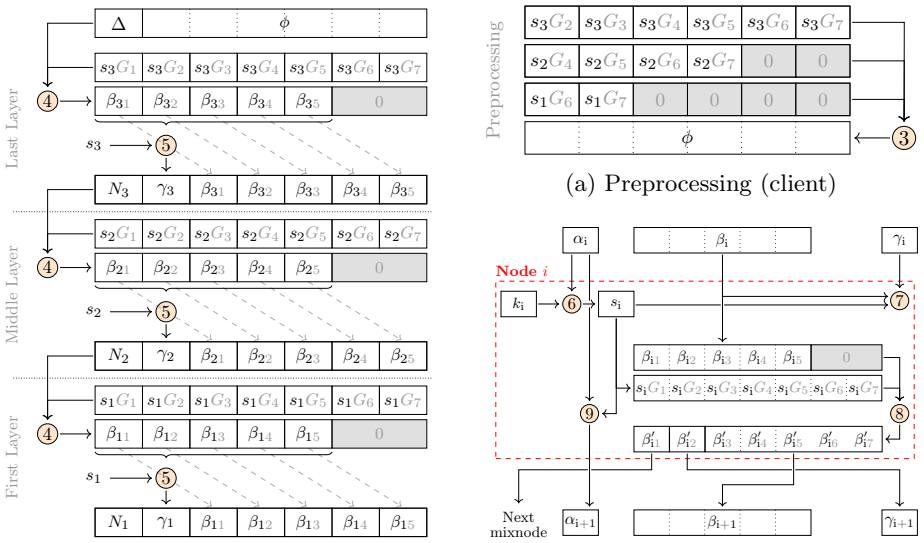
1. **Chunk encryption:** Each chunk j is encrypted according Eq. (4). The last two chunks evaluate to the identity element due to the preprocessing phase and may therefore be discarded while preserving subsequent processing.

$$H_{ij} = H_{(i+1,\,j)} + s_{\mathrm{i}} G_{\mathrm{j}} \tag{4}$$

2. **Integrity computation:** Remaining chunks are used for the integrity tag:

$$\gamma_{\mathrm{i}} = \mathrm{HMAC}(\beta_{\mathrm{i}}, s_i) \tag{5}$$

3. **Prepend Node:** The node address N_{i} and the integrity tag γ_{i} are prepended to H_i, producing again a n-chunks header.



(a) Preprocessing (client)

(b) Encryption (client)

(c) Decryption (network node)

Figure 2: Overview of chunk-wise header processing for a path of length $m = 3$. Circled numbers ⊗ refers to Equation x in the paper.

Algorithm 1 Preprocessing (Fig. 2a)

Input:

$[s_i]_{i=1..m}$	shared secrets
$[G_j]_{j=1..n}$	chunk generators

Output:

ϕ	filler chunks

$m \leftarrow pathLength$
$n \leftarrow 2m + 1$
$\phi \leftarrow$ list of $(n-1)$ identity points
for $i = m$ **down to** 1 **do**
  for $j = 2(m - i + 1)$ **to** n **do**
    $l \leftarrow 2i + j - n$
    $\phi[l] \leftarrow \phi[l] - s_{\mathrm{i}} G_{\mathrm{j}}$
  end for
end for
return ϕ

Algorithm 2 Encryption (Fig. 2b)

Input:

Δ	Destination
$[N_i]_{i=1..m}$	Route

Output:

H	header chunks

$H \leftarrow [\,\Delta \,\|\, \phi\,]$
for $i = m$ **down to** 1 **do**
  for $j = 1$ **to** n **do**
    $H[j] \leftarrow H[j] + s_{\mathrm{i}} G_{\mathrm{j}}$
  end for
  $H \leftarrow H[1 : n-2]$
  $\gamma_{\mathrm{i}} \leftarrow \mathrm{HMAC}(H, s_{\mathrm{i}})$
  $H \leftarrow [\,N_{\mathrm{i}} \,\|\, \gamma_{\mathrm{i}} \,\|\, H\,]$
end for
return H

4.5 Network Nodes Processing

Upon receiving a packet, node i extracts the cryptographic element α_i, the integrity tag γ_i, and the encrypted routing information β_i. As in the original Sphinx design, the node checks whether α_i has been previously observed since its last key rotation and discards duplicates to prevent replay attacks.

The header is processed as illustrated in Fig. 2c. The node first derives the shared secret s_i by multiplying the cryptographic element α_i with its private key k_i, and then hashing the resulting point into a scalar as in (6). The shared secret s_i is then used to verify the integrity tag in (7). If verification succeeds, the node appends two identity chunks and decrypts the header as in (8). The first decrypted chunk $\beta'_{\mathrm{i}1}$ reveals the next hop $N_{\mathrm{i}+1}$. The second chunk $\beta'_{\mathrm{i}2}$ contains the integrity tag $\gamma_{\mathrm{i}+1}$ for the next hop, while the remaining chunks form the next encrypted routing information $\beta_{\mathrm{i}+1}$. Finally, the node updates the cryptographic element according to (9) and forwards the packet to the next hop.

$$s_\mathrm{i} = \mathrm{HashToScalar}(k_\mathrm{i}\,\alpha_\mathrm{i}) \tag{6}$$

$$\gamma_\mathrm{i} \stackrel{?}{=} \mathrm{HMAC}\big(\beta_\mathrm{i}, s_\mathrm{i}\big) \tag{7}$$

$$\beta'_{\mathrm{ij}} = \beta_{\mathrm{ij}} - s_\mathrm{i} G_\mathrm{j} \tag{8}$$

$$\alpha_{\mathrm{i}+1} = s_i\,\alpha_\mathrm{i} \tag{9}$$

5 Security Analysis

This section analyzes the security guarantees of DSphinx and the assumptions under which they hold. We first study packet unlinkability, which constitutes the primary anonymity objective inherited from Sphinx, followed by integrity guarantees and collusion scenarios. We assume that the ECDLP is hard in the underlying group, that hash functions are modeled as random oracles, and that HMAC is existentially unforgeable under chosen-message attacks. We further assume that network nodes anonymously submit their secret shares to the STTPs during the setup phase.

5.1 Unlinkability

To argue about sender-receiver unlinkability, we assume that an adversary cannot link a sender to its recipient without retracing the entire communication path. Thus, the problem reduces to proving unlinkability at the node level, meaning that an adversary cannot determine which outgoing packet corresponds to a given incoming packet. We formalize unlinkability via a standard indistinguishability game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$. Intuitively, the adversary $\mathcal{A}$ observes packets entering and leaving a node and attempts to determine which outgoing packet corresponds to a given incoming packet. The scheme provides unlinkability if any probabilistic polynomial-time adversary achieves only a negligible advantage over random guessing.

Game Definition:

1. **Setup.** The challenger generates the node's secret keys and the public system parameters, including a set of chunk generators $(G_1, \ldots, G_n)$ derived via hash-to-curve. Under the random oracle model, these generators are assumed to be computationally independent.
2. **Oracle.** The adversary is granted access to an oracle, which on input of a nonce w and routing path $\pi = (N_1, \ldots, N_m, \Delta)$ outputs a valid header (α, β, γ) and its updated form $(\alpha', \beta', \gamma')$ after being processed by the node.
3. **Challenge.** The adversary submits two distinct headers $(\alpha_0, \beta_0, \gamma_0)$ and $(\alpha_1, \beta_1, \gamma_1)$ that were not obtained during oracle queries. The challenger selects a random bit $c \in \{0, 1\}$ and processes $(\alpha_c, \beta_c, \gamma_c)$ through the node, obtaining $(\alpha'_c, \beta'_c, \gamma'_c)$, which is then given to the adversary.
4. **Guess.** The adversary outputs a bit $a \in \{0, 1\}$, attempting to guess which input headers corresponds to the given output header. He wins if $a = c$.

Proof Sketch: As with the original Sphinx, all packets have a fixed size, preventing any adversary from correlating packets based on observable size patterns. Consequently, any successful distinguishing strategy must exploit correlations between header components (α, β, γ) across layers. Since the integrity tag γ is computed independently at each layer, it cannot be used for cross-layer linkage.

Case 1: Linking via $\boldsymbol{\alpha}$. Network nodes reject packets with duplicate α values, preventing trivial replay-based linkage. The only relation between α and its processed form α' is the node's shared secret s. Recovering s from $\alpha' = s\alpha$ requires solving an instance of the Elliptic Curve Discrete Logarithm Problem.

Case 2: Linking via $\boldsymbol{\beta}$. Each header chunk β_j and its corresponding processed value β'_j satisfy $\beta_j - \beta'_j = sG_j$. To establish a link, $\mathcal{A}$ must determine whether the same scalar s was used across two chunks. Assuming independent chunk generators, this reduces to solving the ECDLP by either recovering s or computing the discrete logarithmic relation between two generators.

Summary. Under the ECDLP hardness assumption and the independence assumption on the chunk generators, no probabilistic polynomial-time adversary can achieve a non-negligible advantage in the unlinkability game. Therefore, the proposed scheme provides unlinkability.

5.2 Integrity, Replay and Wrap Resistance

Packet integrity relies on a per-hop keyed HMAC computed from the network node shared secret. An adversary modifying any packet header component must therefore produce a valid integrity tag for the modified packet to be accepted by an honest node. Without knowledge of the secret key, forging such a tag is computationally infeasible under standard HMAC security assumptions.

Replay Resistance. As in Sphinx, nodes maintain a record of previously observed values of α. Any packet containing a duplicate value is immediately discarded, preventing replay attacks.

Wrap Resistance. Wrap resistance ensures that an adversary cannot transform a packet (α, β, γ) into $(\alpha', \beta', \gamma')$ that decrypts to the original packet after processing. Such an attack requires modifying a packet while still producing a valid integrity tag. Therefore, wrap resistance reduces to integrity-forgery resistance.

5.3 Collusion Analysis

We now analyze the impact of relevant collusion scenarios between entities involved in the protocol, omitting Client-STTP collusion as it does not add additional capabilities beyond the cases considered below.

Threshold STTP Compromise. The most severe scenario occurs when at least d STTPs collude, where d denotes the reconstruction threshold. In this case, the adversary can reconstruct all network node secrets. This compromises the security of the entire system. Accordingly, the threshold parameter d must be chosen such that collusion of this magnitude remains unrealistic.

STTP - Network Node Collusion. A collusion between an STTP and a network node reveals the mapping between that node's pseudonym and its real identity, resulting in partial deanonymization of the routing space. While a single collusion does not yield a meaningful advantage, multiple colluding nodes may enable partial identification of sampled routes. From the network node's perspective, learning pseudonyms associated with other nodes may enable attempts to optimize its own pseudonym value. Assuming honest nodes choose pseudonyms uniformly at random, any such advantage remains negligible.

Client - Network Node Collusion. Unlike Sphinx, DSphinx intentionally replaces long-term node public keys with a secret-sharing architecture distributed among STTPs. As a consequence, clients cannot independently compute node shared secrets. If a client colludes with a node and learns its secret key, this restriction no longer applies for that node. The client may then bypass the decentralized route sampling mechanism and force inclusion of the colluding node in arbitrary routes. Alternatively, the client may replace legitimate hops with the colluding node after route generation. Although this weakens route sampling guarantees, it does not compromise the secrets of honest nodes and therefore does not break the security of the entire network.

6 Performance Evaluation

We evaluate the computational overhead introduced by DSphinx and its scalability with respect to the two protocol parameters that dominate its computational cost: the path length m and the reconstruction threshold d.

We implemented DSphinx in Python and Rust[2]. The evaluation presented in this section uses the Python prototype, benchmarked on a laptop equipped with a 13th Gen Intel Core i7-13700HX CPU (3.70 GHz) and 32 GB of RAM. The benchmark considers a network of 100 nodes, 50 STTPs, and 40 clients,

[2] `https://github.com/AurelienCha/Decentralized-Sphinx.git`

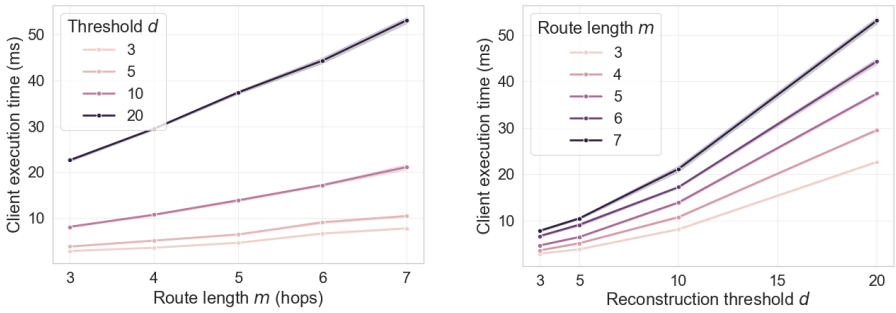


Figure 3: Client-side execution time (ms) of DSphinx, including packet construction and decentralized route sampling, under varying path length (m) and reconstruction threshold (d).

where each client constructs a single DSphinx packet using the decentralized route-sampling protocol. We vary the path length $m \in \{3, 4, 5, 6, 7\}$ and the reconstruction threshold $d \in \{3, 5, 10, 20\}$, resulting in 20 benchmark configurations. Network latency and communication overhead are intentionally excluded so that the measurements isolate the computational cost of the protocol.

Figure 3 reports the total client-side runtime required for packet construction and decentralized route sampling. Depending on the parameter configuration, the runtime ranges from approximately 3 ms to 50 ms, compared to 0.5–2 ms for the original Sphinx. The measured runtimes are consistent with the theoretical complexity of the protocol. In particular, the dominant cost arises from decentralized route sampling, whose threshold reconstruction requires $\mathcal{O}(d \log d)$ operations for each of the m routing hops, yielding an overall client-side complexity of $\mathcal{O}(md \log d)$. This trend is clearly reflected in Figure 3, where increasing either the path length or the reconstruction threshold results in an approximately linear increase in execution time.

Figure 4 summarizes the execution times of the main protocol phases and illustrates their dependence on the path length (m) or reconstruction threshold (d). On the client side, execution is dominated by decentralized route sampling, which accounts for approximately 86% (approximately 2–50 ms) of the total runtime, whereas DSphinx packet construction contributes only the remaining 14% (approximately 0.5–4.5 ms). On the network side, packet processing remains lightweight, requiring only 0.5–0.8 ms per packet, compared with approximately 0.15 ms for Sphinx. Although the node setup phase is the most computationally expensive operation (approximately 15–100 ms), it is performed only once during initialization. Similarly, STTPs introduce only negligible computational overhead. Processing a route-sampling request requires approximately 0.3–0.7 ms, while the one-time setup phase completes in approximately 0.3 ms.

Overall, the evaluation highlights the trade-off introduced by DSphinx. Although decentralized route sampling increases the client-side computational cost

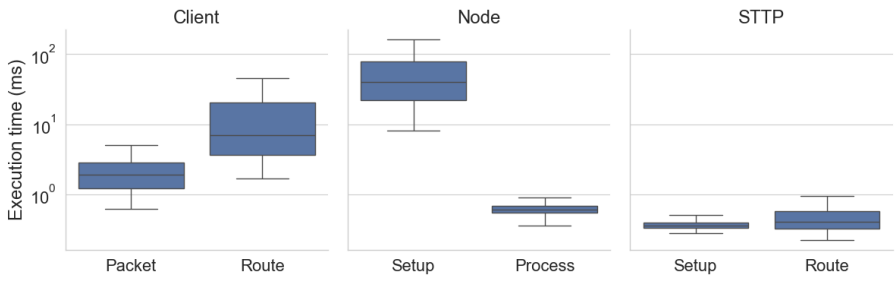


Figure 4: Execution times (ms) for DSphinx client, node, and STTP operations under varying path length m and reconstruction threshold d.

compared with Sphinx, the absolute overhead remains modest in the context of deployed anonymous communication systems. Recent measurements of the Nym mixnet report end-to-end latencies of approximately 500–800 ms, largely dominated by mixing delays (around 50 ms per hop) and network propagation rather than cryptographic processing [12]. In comparison, the additional overhead introduced by DSphinx remains below 50 ms, making it unlikely to dominate end-to-end communication latency in practice.

7 Discussion, Limitations and Future Work

This section discusses the design choices of DSphinx, their implications, and the remaining limitations of the proposed approach.

Impact on route manipulation attacks. DSphinx aims to reduce a malicious client's ability to manipulate routing decisions. However, it does not completely eliminate the possibility of route manipulation attacks. In conventional source-routing systems, clients are free to construct arbitrary routes and repeatedly force the inclusion or exclusion of specific nodes. By contrast, DSphinx derives routes through a decentralized sampling procedure, preventing clients from directly controlling path selection. Although an adversary may repeatedly query the sampling mechanism until obtaining a desirable route, this is substantially more costly than arbitrary route construction. Once a route is sampled, it may be reused multiple times by re-randomizing α and the per-hop shared secrets. Furthermore, repeated reuse of the same sampled route may create observable traffic patterns, making route-manipulation attacks easier to detect.

Limitations and Future Work. DSphinx has few limitations. First, a deployment challenge in establishing a sufficiently large, diverse, and independent set of STTP operators. Second, it relies on a threshold trust assumption: compromise of at least d STTPs breaks header confidentiality and enables full path reconstruction. Third, decentralizing route sampling introduces additional computation and communication overhead. Although modest (less than 50 ms in

our evaluation) compared with the end-to-end latency of decentralized networks such as the Lightning Network and mixnets, these costs should be considered in latency-sensitive settings.

Finally, the current design focuses on *random* route sampling, making it directly applicable to anonymous communication systems such as mixnets, where uniformly random paths are desirable. Extending the approach to routing algorithms based on optimization objectives, such as shortest-path, minimum-cost, or congestion-aware routing used in payment-channel networks like Lightning, remains an open research direction.

8 Conclusion

The main contribution of this paper is DSphinx, a decentralized path sampling scheme designed to prevent malicious clients from deviating from the routing protocol in decentralized privacy networks.

We provide a formal game-based security analysis of unlinkability, a core security notion in privacy-preserving communication systems. Furthermore, we analyze integrity, replay resistance, and wrap resistance under both passive and active network adversaries, corrupted network nodes, and colluding STTPs. We show that DSphinx preserves the cryptographic guarantees of Sphinx, provided that fewer than a threshold number d of STTPs collude.

We also implement a prototype and evaluate its performance. The results show that DSphinx introduces a latency overhead of less than 50 ms, which remains acceptable compared to the end-to-end latency of typical decentralized networks (approximately 500-800 ms in mixnets). In terms of scalability, we show that the protocol runs in $\mathcal{O}(md \log d)$ time, where m denotes the number of hops in the route and d the reconstruction threshold of STTPs.

Overall, DSphinx preserves the core privacy and integrity guarantees of Sphinx while reducing reliance on centralized components, thereby representing a step toward more robust and flexible decentralized privacy networks.

Acknowledgments. We gratefully acknowledge the Brussels-Capital Region – Innoviris for financial support under grant numbers 2024-RPF-2 MImPG and 2024-RPF-4 SDM, and the CyberExcellence program of the Walloon Region of Belgium under grant number 2110186. Iness Ben Guirat is a Postdoctoral Researcher of the Fonds de la Recherche Scientifique – FNRS.

References

1. ACINQ: Eclair: A Scala Implementation of the Lightning Network (2026)
2. Alexopoulos, N., Kiayias, A., Talviste, R., Zacharias, T.: Mcmix: Anonymous messaging via secure multiparty computation. In: 26th USENIX Security Symposium (USENIX Security 17) (2017)
3. Ben Guirat, I., Diaz, C.: Mixnet optimization methods. Proceedings on Privacy Enhancing Technologies (2022)

4. Ben Guirat, I., Gosain, D., Diaz, C.: Mixim: Mixnet design decisions and empirical evaluation. In: Proceedings of the 20th Workshop on Workshop on Privacy in the Electronic Society. pp. 33–37 (2021)
5. Bernstein, D.J., Hamburg, M., Krasnova, A., Lange, T.: Elligator: elliptic-curve points indistinguishable from uniform random strings. In: Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security (2013)
6. Chaum, D.: Untraceable electronic mail, return addresses, and digital pseudonyms. Commun. ACM **24**(2), 84–88 (1981)
7. Chaum, D., Javani, F., Kate, A., Krasnova, A., Ruiter, J., Sherman, A.T., Das, D.: cmix: Anonymization by high-performance scalable mixing (2016)
8. Cottrell, L.: Mixmaster and remailer attacks (1995)
9. Danezis, G., Dingledine, R., Mathewson, N.: Mixminion: Design of a Type III Anonymous Remailer Protocol. In: Proceedings of the 2003 IEEE Symposium on Security and Privacy. IEEE (2003)
10. Danezis, G., Goldberg, I.: Sphinx: A compact and provably secure mix format. In: 2009 30th IEEE Symposium on Security and Privacy (2009)
11. Diaz, C., Halpin, H., Kiayias, A.: The nym network (2021)
12. Diaz, C., Halpin, H., Kiayias, A.: Decentralized reliability estimation for low latency mixnets (2024)
13. Elements Project: c-lightning: A Lightning Network Implementation in C (2026)
14. Hooff, J.V.D., Lazar, D., Zaharia, M., Zeldovich, N.: Vuvuzela: Scalable private messaging resistant to traffic analysis. In: Proceedings of the 25th Symposium on Operating Systems Principles (2015)
15. Infeld, E.J., Stainton, D., Ryge, L., Hacker, T.: Echomix: a strong anonymity system with messaging (2025)
16. Kappos, G., Yousaf, H., Piotrowska, A., Kanjalkar, S., Delgado-Segura, S., Miller, A., Meiklejohn, S.: An empirical analysis of privacy in the lightning network. In: International Conference on Financial Cryptography and Data Security. pp. 167–186. Springer (2021)
17. Kumble, S.P., Epema, D., Roos, S.: How lightning's routing diminishes its anonymity. In: Proceedings of the 16th International Conference on Availability, Reliability and Security. pp. 1–10 (2021)
18. Kwon, A., Lu, D., Devadas, S.: XRD: Scalable messaging system with cryptographic privacy. In: 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20) (2020)
19. Lazar, D., Gilad, Y., Zeldovich, N.: Karaoke: Distributed private messaging immune to passive traffic analysis. In: 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18) (2018)
20. Lightning Network Developers: LND: Lightning Network Daemon (2026)
21. Piotrowska, A.M., Hayes, J., Elahi, T., Meiser, S., Danezis, G.: The loopix anonymity system. In: 26th USENIX Security Symposium (USENIX Security 17) (2017)
22. Poon, J., Dryja, T.: The bitcoin lightning network: Scalable off-chain instant payments (2016)
23. Rohrer, E., Malliaris, J., Tschorsch, F.: Discharged payment channels: Quantifying the lightning network's resilience to topology-based attacks. In: 2019 IEEE European Symposium on Security and Privacy Workshops. pp. 347–356 (2019)
24. Sharma, P.K., Gosain, D., Diaz, C.: On the anonymity of peer-to-peer network anonymity schemes used by cryptocurrencies (2022)
25. Tochner, S., Schmid, S., Zohar, A.: Hijacking routes in payment channel networks: A predictability tradeoff. arXiv preprint arXiv:1909.06890 (2019)